

Recomandarea produselor: abordari bazate pe Machine Learning

Sistem de recomandare pentru filme bazat pe
algoritmi de invatare automata

Vint Alexandru

Mai 2026

Disciplina: Sisteme Inteligente
Set de date: MovieLens (ml-latest-small)

Cuprins

1	Introducere	4
1.1	Obiectivul proiectului	4
1.2	Tehnologii utilizate	4
1.3	Structura proiectului	4
2	Setul de date	4
2.1	Sursa datelor	4
2.2	Continutul setului de date	4
2.3	Fisierele utilizate	5
3	Preprocesarea datelor	5
3.1	Tratarea valorilor lipsa pentru genuri	5
3.2	Extragerea anului de lansare	5
3.3	Conversia timestamp-urilor si normalizarea tag-urilor	6
4	Analiza exploratorie a datelor (EDA)	6
4.1	Distributia rating-urilor	6
4.2	Statistici pe utilizatori si filme	6
4.3	Matricea de corelatie	6
5	Feature Engineering	7
5.1	Statistici agregate	7
5.2	One-hot encoding pentru genuri	7
5.3	Features temporale	8
5.4	User-genre affinity	8
6	Impartirea datelor train/test	8
7	Modele de recomandare	8
7.1	k-NN Item-Based (filtrare colaborativa)	9
7.2	k-NN User-Based	9
7.3	k-NN Content-Based cu TF-IDF	9
7.4	Random Forest Regressor	10
7.5	XGBoost Regressor	10
7.6	Matrix Factorization (SGD)	10
7.7	Deep Learning: Neural Collaborative Filtering (NCF)	11
8	Rezultate si comparatie	11
8.1	Discutie	12
9	Analiza modelelor (Explainability)	12
9.1	Feature importance pentru modele tree-based	12
9.2	Explainability pentru content-based (TF-IDF)	13
9.3	Limitarile interpretabilitatii pentru modele colaborative	13
9.4	Concluzii ale analizei	13
10	Concluzii	13
10.1	Directii viitoare	14

1 Introducere

Sistemele de recomandare au devenit o componenta esentiala a platformelor digitale moderne (Netflix, YouTube, Amazon, Spotify), influentand semnificativ atat experienta utilizatorilor, cat si performanta economica a serviciilor respective. Cantitatea masiva de continut disponibil face imposibila explorarea manuala a tuturor optiunilor, motiv pentru care algoritmi de recomandare automatizeaza procesul de filtrare si sugerare a elementelor relevante pentru fiecare utilizator.

1.1 Obiectivul proiectului

Obiectivul acestui proiect este dezvoltarea unui sistem de recomandare pentru filme care sugereaza utilizatorilor titluri in functie de preferintele si istoricul lor de vizionare, folosind tehnici diverse de *Machine Learning*. Sunt implementate, evaluate si comparate sapte abordari distincte, acoperind atat metode clasice (filtrare colaborativa bazata pe vecini, modele bazate pe continut), cat si metode moderne (matrix factorization, deep learning).

1.2 Tehnologii utilizate

- **Limbaj:** Python 3
- **Manipulare date:** pandas, NumPy
- **Modele clasice ML:** scikit-learn (k-NN, Random Forest, TF-IDF, metrice)
- **Gradient boosting:** XGBoost
- **Deep Learning:** PyTorch
- **Vizualizare:** Matplotlib, Seaborn
- **Mediu de dezvoltare:** Jupyter Notebook

1.3 Structura proiectului

- `src/python.ipynb` – notebook-ul principal cu toate experimentele
- `src/dataset/` – setul de date MovieLens (`ratings.csv`, `movies.csv`, `tags.csv`, `links.csv`)
- `documentatie/` – raportul de proiect
- `screenshots/` – capturi de ecran cu rezultate intermediare

2 Setul de date

2.1 Sursa datelor

Datele provin din proiectul **MovieLens** dezvoltat de grupul de cercetare GroupLens din cadrul Departamentului de Informatica al Universitatii din Minnesota. A fost folosita versiunea *ml-latest-small*, descarcata in data de 17 martie 2026 de la adresa <https://grouplens.org/datasets/movielens/latest/>.

2.2 Continutul setului de date

Setul de date contine:

- **100.836** de rating-uri
- **3.683** de tag-uri textuale

- **9.742** de filme
- **610** utilizatori (fiecare cu cel puțin 20 de rating-uri)

2.3 Fisierele utilizate

Fisier	Coloane	Descriere
<code>ratings.csv</code>	<code>userId</code> , <code>movieId</code> , <code>rating</code> , <code>timestamp</code>	Rating-uri 0.5–5.0 stele
<code>movies.csv</code>	<code>movieId</code> , <code>title</code> , <code>genres</code>	Titlu + genuri (pipe-separated)
<code>tags.csv</code>	<code>userId</code> , <code>movieId</code> , <code>tag</code> , <code>timestamp</code>	Tag-uri text aplicate de utilizatori
<code>links.csv</code>	<code>movieId</code> , <code>imdbId</code> , <code>tmdbId</code>	Identificatori externi

Tabela 1: Structura setului de date MovieLens

3 Preprocesarea datelor

3.1 Tratarea valorilor lipsa pentru genuri

Setul de date MovieLens vine initial cu doua conventii specifice pentru coloana `genres`: filmele care au mai multe genuri le au listate intr-un singur string, separate prin caracterul `"|"` (de exemplu, `Adventure|Comedy|Drama`), iar filmele care nu au niciun gen asociat sunt marcate cu valoarea (`no genres listed`).

In etapa de preprocesare am realizat doi pasi:

1. Am eliminat valorile (`no genres listed`) din coloana `genres`, pentru a curata marcasele placeholder din setul de date.
2. Am impartit genurile dupa caracterul `|`, obtinand o coloana noua `genres_list` in care fiecare film este reprezentat ca o lista de stringuri cu genurile sale.

```

1 movies['genres'] = movies['genres'].replace('no genres listed', '')
2 movies['genres_list'] = movies['genres'].apply(
3     lambda x: x.split('|') if x != '' else []
4 )

```

3.2 Extragerea anului de lansare

Anul de lansare al fiecarui film este prezent in titlu, intre paranteze. A fost extras cu o expresie regulata si mutat intr-o coloana separata `year`:

```

1 movies['year'] = movies['title'].str.extract(r'\((\d{4})\)').astype(
2     float)
3 movies['title'] = movies['title'].str.replace(r'\s*(\d{4})', '', regex
4     =True)

```

Pentru cele 13 filme la care anul nu a putut fi extras automat din titlu, am identificat manual anul corect al fiecaruia si l-am introdus intr-un dictionar `year_map`, folosit ulterior pentru completarea valorilor lipsa.

3.3 Conversia timestamp-urilor si normalizarea tag-urilor

Coloanele `timestamp` din `ratings` si `tags` au fost convertite din *Unix epoch* in obiecte `datetime` pentru a permite extragerea de features temporale. Tag-urile au fost normalizate la minuscule, eliminandu-se spatiile inutile.

4 Analiza exploratorie a datelor (EDA)

4.1 Distributia rating-urilor

Rating-urile sunt concentrate spre valorile mari ale scalei: utilizatorii tind sa acorde note mai mari decat cele mici. Valoarea cea mai frecventa este **4.0**, urmata de **3.0** si **5.0**. Rating-urile cu jumatati de stea (0.5, 1.5, 2.5, etc.) sunt semnificativ mai rare.

4.2 Statistici pe utilizatori si filme

- Numarul de rating-uri pe utilizator are o distributie concentrata spre valori mici, cu cateva valori extreme: cei mai activi utilizatori au cateva mii de rating-uri, in timp ce mediana este mult mai mica.
- Distributia numarului de rating-uri pe film este puternic dezechilibrata: cateva filme populare acumuleaza majoritatea voturilor, iar majoritatea filmelor au putine rating-uri.

4.3 Matricea de corelatie

Analiza corelatiilor intre variabilele numerice (`rating`, `year`, `avg_rating`, `num_ratings`, `user_avg_rating`, `user_num_ratings`) arata ca cei mai puternici predictori pentru **rating** sunt:

- `avg_rating` – media rating-urilor primite de film
- `user_avg_rating` – media rating-urilor date de utilizator

Aceste doua variabile vor sta la baza modelelor supervizate (Random Forest, XGBoost).

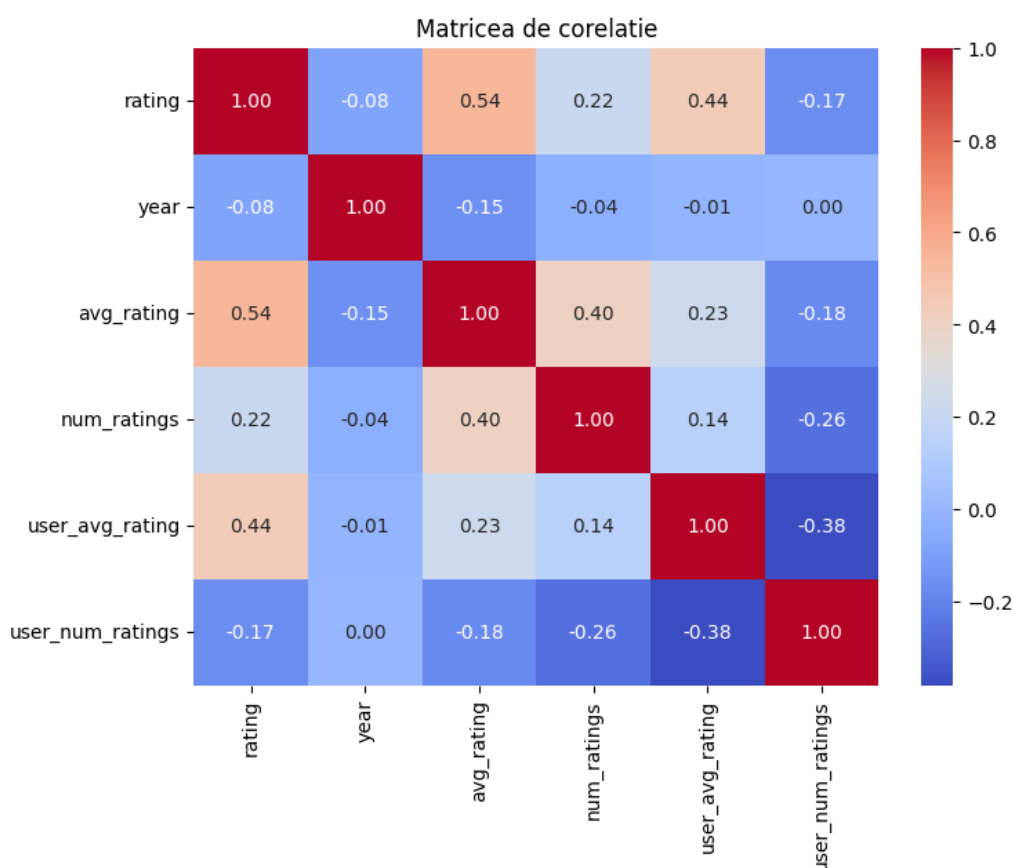


Figura 1: Matricea de corelatie intre variabilele numerice ale setului de date.

5 Feature Engineering

Pe langa variabilele brute, au fost construite mai multe features derivate care imbogatesc reprezentarea perechilor (user, film):

5.1 Statistici agregate

- **Per film:** `avg_rating`, `num_ratings`

Combinatia acestor doua features ajuta modelul sa identifice filmele bune si populare. De exemplu, un film cu `avg_rating` = 4.5 si `num_ratings` = 1000 este probabil un film de calitate cu impact semnificativ, in timp ce un `avg_rating` mare bazat pe doar cateva voturi este nesemnificativ statistic.

- **Per utilizator:** `user_avg_rating`, `user_num_ratings`

Aceste features captureaza „personalitatea” fiecarui utilizator – tendinta sa de a fi mai generos sau mai sever in evaluari. De exemplu, un utilizator cu `user_avg_rating` = 4.37 calculat pe 232 de rating-uri tinde sa fie un evaluator generos, iar modelul foloseste acest context pentru a-i calibra predictiile.

5.2 One-hot encoding pentru genuri

Am binarizat cele sapte genuri cele mai populare (Action, Comedy, Drama, Horror, Romance, Sci-Fi, Thriller), creand cate o coloana binara pentru fiecare: `is_Action`,

is_Comedy, is_Drama, etc. Fiecare coloana ia valoarea 1 daca filmul apartine genului respectiv si 0 in caz contrar.

5.3 Features temporale

Din timestamp-ul rating-ului au fost extrase:

- **rating_year** – anul in care a fost acordat rating-ul, util pentru a surprinde tendinte de lunga durata (ex.: filme mai vechi sunt evaluate diferit fata de cele noi).
- **rating_month** si **rating_dayofweek** – captureaza posibile pattern-uri sezoniere si comportamente recurente (ex.: filme de actiune in weekend, filme romantice in februarie).
- **years_since_release** = **rating_year** – **year** – numarul de ani scursi intre lansarea filmului si momentul evaluarii sale. Aceasta este probabil cea mai informativa features temporala: un film evaluat la 20 de ani dupa lansare este vazut de regula de fani sau cunoscatori, ceea ce explica adesea rating-uri sistematic mai mari.

5.4 User-genre affinity

Cea mai informativa caracteristica derivata: media rating-urilor pe care un utilizator le-a acordat filmelor dintr-un anumit gen. Operatia consta in transformarea coloanei **genres_list** astfel incat fiecare gen sa apara pe un rand separat, urmata de un pivot **userId × genre** cu agregare **mean**. Pentru combinatiile lipsa (utilizatori care nu au votat un gen) s-a folosit media generala a utilizatorului ca valoare default.

6 Impartirea datelor train/test

Setul de rating-uri a fost impartit aleator in **80% antrenament** si **20% testare**, cu **random_state=42** pentru reproductibilitate.

```
1 train_ratings, test_ratings = train_test_split(  
2     ratings_cf, test_size=0.2, random_state=42  
3 )
```

Pentru evaluarea modelelor s-au folosit doua metrici standard:

- **RMSE** (Root Mean Squared Error):

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{r}_i - r_i)^2}$$

- **MAE** (Mean Absolute Error):

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\hat{r}_i - r_i|$$

7 Modele de recomandare

In continuare sunt prezentate cele sapte abordari implementate.

7.1 k-NN Item-Based (filtrare colaborativa)

Ideea: doua filme sunt similare daca au primit rating-uri asemanatoare de la aceiasi utilizatori. Pentru a recomanda filme similare unui film m , se gasesc cei mai apropiati k vecini ai sai intr-un spatiu vectorial.

Implementare:

1. S-a construit matricea sparse $\text{movieId} \times \text{userId}$ cu valori rating.
2. S-a antrenat **NearestNeighbors** cu metrica **cosine** si algoritmul **brute**.
3. Pentru un film dat, se obtin top- k vecini, excluzand filmul insusi.

Recomandarea returneaza lista de filme cu cele mai mici distante cosinus.

7.2 k-NN User-Based

Ideea: pentru a recomanda filme unui utilizator, algoritmul identifica mai intai cei mai similari k utilizatori pe baza istoricului lor de rating-uri si recomanda apoi filmele preferate de acestia, pe care utilizatorul tinta nu le-a vizionat inca.

Algoritm:

1. Construire matrice $\text{userId} \times \text{movieId}$.
2. Calcul vecini cu cosine similarity.
3. Pentru fiecare film nevizionat de u :

$$\hat{r}_{u,m} = \frac{\sum_{v \in N(u)} \text{sim}(u, v) \cdot r_{v,m}}{\sum_{v \in N(u)} \text{sim}(u, v)}$$

4. Sortare descrescatoare dupa scor, returnare top- N .

7.3 k-NN Content-Based cu TF-IDF

Ideea: doua filme sunt similare daca tag-urile lor textuale sunt similare. Spre deosebire de abordarile colaborative, aceasta metoda **nu foloseste rating-urile**, ci doar continutul (tag-urile aplicate de utilizatori).

Implementare:

1. Agregarea tag-urilor per film: fiecare film devine un „document” textual.
2. Vectorizare TF-IDF cu $\text{min_df}=2$ (ignora tag-uri foarte rare) si $\text{max_df}=0.8$ (ignora tag-uri foarte comune).
3. Antrenare k-NN cu metrica cosine pe matricea TF-IDF.

Formula TF-IDF (pentru tag t in film m):

$$\text{tfidf}(t, m) = \text{tf}(t, m) \cdot \log \frac{N}{\text{df}(t)}$$

Aceasta metoda ofera explicabilitate excelenta: pentru fiecare recomandare se pot afisa tag-urile comune cu filmul sursa.

7.4 Random Forest Regressor

Ideea: problema este transformata din *recomandare* in *regresie supervizata*. Pentru fiecare pereche (user, film) din setul de antrenament se prezice rating-ul folosind un ansamblu de arbori de decizie.

Configuratie finala (v3):

- Features: avg_rating, num_ratings, user_avg_rating, user_num_ratings, year, 7 features binare pentru genuri, 3 features temporale, 7 features de tip affinity_<gen>.
- Hiperparametri: n_estimators=100, max_depth=10.

Recomandarea se genereaza evaluand modelul pe toate filmele nevizionate de un utilizator si sortand descrescator dupa rating-ul prezis. S-a aplicat un filtru suplimentar num_ratings >= 20 pentru a evita filmele obscure cu rating-uri inflationiste.

7.5 XGBoost Regressor

XGBoost este o implementare optimizata a algoritmului *gradient boosting*, in care fiecare arbore corecteaza erorile arborilor anteriori. Comparativ cu Random Forest:

- Arborii sunt construiti **secvential**, nu independent.
- Optimizarea foloseste gradient descent pe o functie obiectiv diferentiabila.
- Regularizarea L1/L2 este integrata in functia obiectiv.

Hiperparametri folositi: n_estimators=150, max_depth=6, learning_rate=0.1.
Features: features_extended (fara affinity_<gen>).

7.6 Matrix Factorization (SGD)

Ideea: matricea rating-urilor $R \in \mathbb{R}^{|U| \times |M|}$ este aproximata prin produsul a doua matrice cu rang mic:

$$R \approx PQ^T + \mu + b_u + b_i$$

unde:

- $P \in \mathbb{R}^{|U| \times k}$ – matricea factorilor latenti pentru utilizatori
- $Q \in \mathbb{R}^{|M| \times k}$ – matricea factorilor latenti pentru filme
- μ – media globala
- b_u, b_i – bias-uri pentru utilizatori si filme

Antrenare prin SGD: parametrii se actualizeaza iterativ minimizand functia de cost regularizata:

$$\mathcal{L} = \sum_{(u,i,r) \in \mathcal{T}} (r_{ui} - \hat{r}_{ui})^2 + \lambda (\|P_u\|^2 + \|Q_i\|^2 + b_u^2 + b_i^2)$$

Regulile de actualizare (pentru fiecare exemplu de antrenament):

$$\begin{aligned} e_{ui} &= r_{ui} - \hat{r}_{ui} \\ P_u &\leftarrow P_u + \eta(e_{ui} \cdot Q_i - \lambda P_u) \\ Q_i &\leftarrow Q_i + \eta(e_{ui} \cdot P_u - \lambda Q_i) \\ b_u &\leftarrow b_u + \eta(e_{ui} - \lambda b_u) \\ b_i &\leftarrow b_i + \eta(e_{ui} - \lambda b_i) \end{aligned}$$

Hiperparametri folositi: `n_factors=50`, `lr=0.005`, `reg=0.02`, `n_epochs=15`.

Nota importanta: implementarea initiala folosea SVD trunchiat din `scipy.sparse.linalg.svds`, dar a fost inlocuita cu o implementare SGD personalizata, mult mai stabila pe matrici extrem de sparse (RMSE: $3.16 \rightarrow 0.88$).

7.7 Deep Learning: Neural Collaborative Filtering (NCF)

Ideea: in loc sa folosim produs scalar liniar intre factorii latenti (ca in MF), folosim o retea neuronală pentru a invata o functie de interactiune non-liniara intre user si film.

Arhitectura:

1. **Embedding-uri:** `user_embedding` si `item_embedding`, fiecare de dimensiune 50.
2. **Concatenare:** vectorul de input este `[user_emb; item_emb]` de dimensiune 100.
3. **MLP:** trei straturi fully-connected ($100 \rightarrow 64 \rightarrow 32 \rightarrow 16$), cu activare ReLU si dropout ($p=0.2$).
4. **Strat de iesire:** liniar ($16 \rightarrow 1$).

Configuratie antrenare:

- Optimizator: Adam, `lr=0.005`, `weight_decay=1e-5` (L2)
- Functie de cost: MSE
- Batch size: 256
- Epoci: 10

Evolutia loss-ului pe parcursul antrenarii:

Epoca	Loss MSE
1	1.5035
3	0.7865
5	0.6898
7	0.6306
10	0.5382

Tabela 2: Evolutia erorii pe parcursul antrenarii NCF

8 Rezultate si comparatie

Toate modelele bazate pe regresie au fost evaluate pe acelasi split train/test (80/20, `random_state=42`).

Model	RMSE	MAE
XGBoost	0.7790	0.5870
Random Forest (v3, cu affinity)	0.7829	0.5905
Matrix Factorization (SGD)	0.8820	0.6748
Deep Learning (NCF)	0.9071	0.7124

Tabela 3: Comparatia performantei modelelor de regresie

8.1 Discutie

XGBoost obtine cel mai bun scor, fiind urmat foarte aproape de Random Forest v3. Diferenta mica (0.004 RMSE) intre cele doua sugereaza ca, pentru acest set de date, contributia suplimentara a boosting-ului fata de bagging este modesta. Ambele modele beneficiaza din plin de **feature engineering-ul** manual (in special `avg_rating` si `user_avg_rating`, care domina feature importance-ul).

Matrix Factorization ofera un rezultat decent (RMSE 0.88) folosind exclusiv informatia din matricea de rating-uri, fara features derivate. Acest lucru evidentiaza puterea metodelor pure de filtrare colaborativa pe seturi de date mari si dense.

NCF obtine cel mai slab scor in acest experiment. Posibile cauze:

- Setul de date este relativ mic (100k rating-uri) pentru a antrena o retea cu sute de mii de parametri.
- Doar 10 epoci pot fi insuficiente; loss-ul de antrenament continua sa scada.
- Lipsa unei strategii de *early stopping* sau validare pe un subset separat.

Modelele k-NN (item-based, user-based, content-based) genereaza direct recomandari fara un scor de rating prezis, deci nu apar in tabelul comparativ cu RMSE/MAE. Ele ofera in schimb explicabilitate (vecinii pot fi inspectati) si sunt utile pentru **cold-start** (in special content-based, care nu necesita rating-uri).

9 Analiza modelelor (Explainability)

Dincolo de metricile cantitative (RMSE, MAE), un aspect esential al unui sistem de recomandare este *explainability-ul* – capacitatea de a intelege ce factori influenteaza deciziile modelului. In aceasta sectiune analizam ce features sunt cele mai informative pentru fiecare clasa de modele si discutam limitarile interpretabilitatii.

9.1 Feature importance pentru modele tree-based

Atat XGBoost cat si Random Forest ofera valori de *feature importance*, care indica cat de mult contribuie fiecare feature la predictiile modelului. Observatiile principale sunt:

- Pentru ambele modele, `avg_rating` (media rating-urilor primite de film) este de departe cel mai important predictor, explicand singura o mare parte din variatia rating-urilor.
- `user_avg_rating` (media rating-urilor acordate de utilizator) ocupa locul al doilea, capturand tendinta utilizatorului de a fi mai generos sau mai sever in evaluari.
- In Random Forest v3, features-urile de tip user-genre affinity (`affinity_Drama`, `affinity_Action`, `affinity_Comedy`) intra in top 5, confirmand utilitatea acestui feature engineering.
- Features-urile temporale (`rating_year`, `years_since_release`) au impact moderat, dar masurabil.
- Features-urile binare pentru genuri (`is_Action`, `is_Comedy`, etc.) au importanta individuala scazuta, fiind dominate de echivalentele lor de tip affinity.

9.2 Explainability pentru content-based (TF-IDF)

Modelul k-NN content-based ofera **explainability nativa**: pentru fiecare recomandare se pot afisa explicit tag-urile comune cu filmul sursa, oferind o justificare interpretabila de tipul „acest film este recomandat pentru ca impartaseste tag-urile X, Y si Z cu filmul pe care l-ati apreciat”. Aceasta este o calitate importanta pentru aplicatii reale, unde utilizatorii beneficiaza de transparenta in procesul de recomandare.

9.3 Limitarile interpretabilitatii pentru modele colaborative

Matrix Factorization si Neural Collaborative Filtering invata reprezentari latente (*embeddings*) dificil de interpretat semantic. Desi factorii latenti captureaza informatie utila (gen implicit, ton, regizor preferat), acestia nu pot fi mapati direct la concepte umane interpretabile, fara analize suplimentare. Aceasta este o limitare cunoscuta a abordarilor pure de filtrare colaborativa si reprezinta un trade-off intre performanta si transparenta.

9.4 Concluzii ale analizei

Analiza arata ca:

1. Modelele cu cel mai bun RMSE (XGBoost, RF) datoreaza performanta in mare parte features-urilor inginerite manual (`avg_rating`, `user_avg_rating`, `affinity`).
2. Modelele cu cea mai buna explainability (content-based) nu sunt neaparat cele mai performante.
3. Modelele de tip embedding (MF, DL) sacrifica interpretabilitatea pentru flexibilitate, fiind potrivite pentru scalare pe date mari.

In contextul unui sistem real, alegerea modelului ar trebui sa balanseze performanta, scalabilitatea si transparenta – iar o abordare hibrida care combina mai multe modele este adesea solutia optima.

10 Concluzii

Proiectul a explorat o gama larga de tehnici pentru construirea unui sistem de recomandare de filme, de la metode clasice (k-NN, TF-IDF) pana la abordari moderne (Matrix Factorization, Neural Collaborative Filtering). Principalele observatii sunt:

1. **Feature engineering-ul** conteaza semnificativ: cele mai performante modele (XGBoost, RF) folosesc features derivate manual (`avg_rating`, `user_avg_rating`, `user-genre affinity`).
2. **Modelele simple performeaza bine**: pe un set de date de dimensiuni modeste, modelele supervizate clasice depasesc abordarile bazate pe deep learning.
3. **Alegerea metricii potrivite** este cruciala: RMSE si MAE masoara precizia predictiei rating-urilor, dar nu reflecta neaparat calitatea ordonarii recomandarilor (Top-N) percepute de utilizator.
4. **Diversitatea abordarilor** are valoare practica: intr-un sistem real, k-NN content-based ar rezolva problema cold-start (filme noi fara rating-uri), MF/NCF ar acoperi cazul general, iar XGBoost ar oferi predictii fine de rating.

10.1 Directii viitoare

- Evaluare pe metrici Top-N (Precision@K, Recall@K, NDCG).
- Implementare *hybrid recommender* care combina predictiile mai multor modele.
- Tuning extins de hiperparametri (Optuna / GridSearchCV).
- Strategii de *early stopping* si *learning rate scheduling* pentru NCF.
- Integrarea explicita a tag-urilor TF-IDF in features-urile XGBoost.

Bibliografie

1. F. Maxwell Harper si Joseph A. Konstan, „The MovieLens Datasets: History and Context”, *ACM Transactions on Interactive Intelligent Systems*, vol. 5, nr. 4, 2015.
<https://doi.org/10.1145/2827872>
2. Y. Koren, R. Bell, C. Volinsky, „Matrix Factorization Techniques for Recommender Systems”, *IEEE Computer*, 2009.
3. X. He et al., „Neural Collaborative Filtering”, *Proceedings of WWW 2017*.
4. T. Chen, C. Guestrin, „XGBoost: A Scalable Tree Boosting System”, *KDD 2016*.
5. L. Breiman, „Random Forests”, *Machine Learning*, vol. 45, 2001.
6. Documentatia scikit-learn: <https://scikit-learn.org/stable/>
7. Documentatia PyTorch: <https://pytorch.org/docs/>